

contention, it's often more prudent to let the Service stop and retry at the next scheduled interval.

- `START_REDELIVER_INTENT` is used in circumstances you'll want to ensure that the commands you have requested from your Service are completed.
- This mode is a combination of the first two—if the Service is terminated by run time, it will restart only if there are pending start calls or the process was killed prior to its calling `stopSelf`.
- In the latter case, a call to `onStartCommand` will be made, passing the initial Intent whose processing did not properly complete.

The restart mode you specify in your `onStartCommand` return value will affect the parameter values passed to subsequent calls. Initially the Intent will be the parameter you passed to `startService` to start your Service. After system-based restarts it will be either null, in the case of `START_STICKY` mode, or the original Intent, if the mode is set to `START_REDELIVER_INTENT`. Use the flag parameter to discover how the Service was started. In particular, you can use the code snippet below to determine if either of the following cases is true:

- `START_FLAG_REDELIVERY` indicates that the Intent parameter is a redelivery caused by the system run time's having terminated the Service before it was explicitly stopped by a call to `stopSelf`.
- `START_FLAG_RETRY` indicates that the Service has been restarted after an abnormal termination. Passed when the Service was previously set to `START_STICKY`.

@Override

```
public int onStartCommand(Intent intent, int flags, int
startId) {
    if ((flags & START_FLAG_RETRY) == 0) {
        // TODO If it's a restart, do something.
    }
    else {
        // TODO Alternative background process.
    }
    return Service.START_STICKY;
}
```

5.2.2 Registering a Service in the Manifest

Once you've constructed a new Service, you must register it in the application manifest. Do this by including a `<service>` tag within the application node.